

SEED Labs: BGP Exploration and Attack Report

Hadji Youcef Housseem Eddine
Aidi Youcef Abdelkader
Zernouh Lakhdar

April,2026

Contents

1 Overview	2
2 Lab Environment Setup	2
2.1 Starting the Emulator	2
2.2 Accessing the Network Map	2
3 Task 1: Stub Autonomous System	3
3.1 Task 1.a: Understanding AS-155's BGP Configuration	3
3.1.1 Task 1.a.1: Identifying AS-155's Peers	3
3.1.2 Task 1.a.2: Demonstrating Redundancy via Failover	3
3.2 Task 1.b: Observing BGP UPDATE Messages	5
3.3 Task 1.c: Experimenting with Large Communities	6
3.4 Task 1.d: Configuring AS-180	7
4 Task 2: Transit Autonomous System	8
4.1 Task 2.a: Experimenting with IBGP	8
4.2 Task 2.b: Experimenting with IGP (OSPF)	10
4.3 Task 2.c: Configuring AS-5	11
4.3.1 IBGP Configuration (Existing)	11
4.3.2 EBGP Peering Configuration	12
5 Task 3: Path Selection	14
5.1 Task 3.a: Identifying the Best Route	14
5.2 Task 3.b: Preferring AS-3 over AS-2	15
6 Task 4: IP Anycast	16
7 Task 5: BGP Prefix Hijacking	17
7.1 Task 5.a: Launching the Attack from AS-161	17
7.2 Task 5.b: Fighting Back from AS-154	18
7.3 Task 5.c: Fixing the Problem at AS-3	20
8 References	20

1 Overview

In this lab, we utilize the SEED Internet Emulator to visualize, configure, and attack BGP infrastructures. The primary objectives are:

- Understanding BGP table structures and route selection.
- Configuring BGP peering (eBGP) between different Autonomous Systems.
- Implementing BGP filtering and community-based routing.
- Demonstrating and mitigating BGP Prefix Hijacking attacks.

2 Lab Environment Setup

The lab is hosted on a Docker-based emulator. The network topology consists of multiple Autonomous Systems (AS-150, AS-151, AS-152, etc.) interconnected via BGP routers.

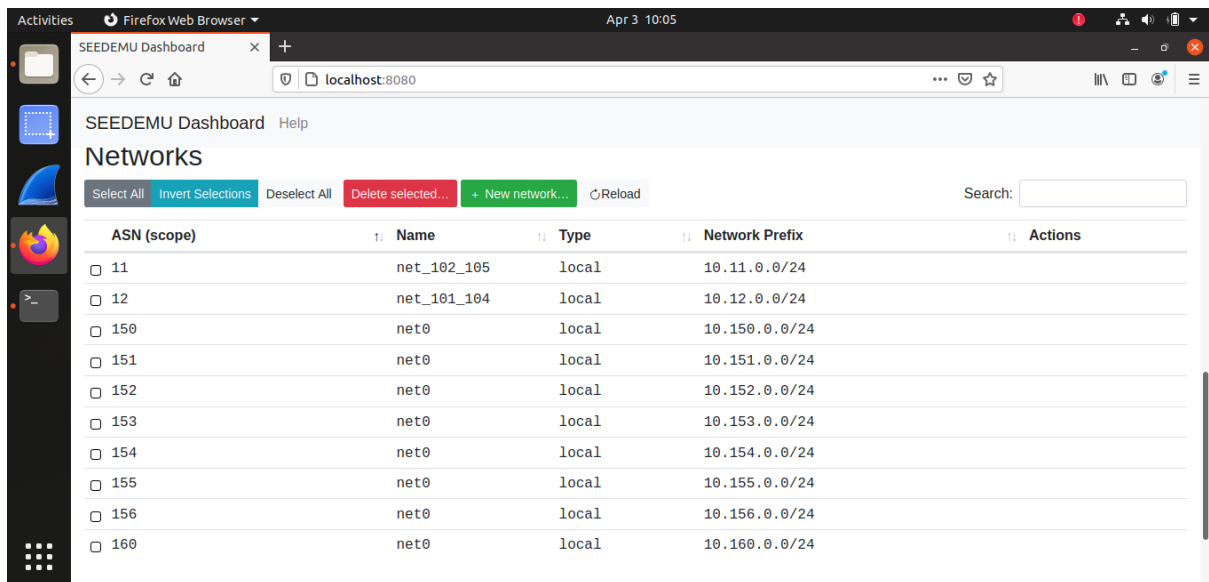
2.1 Starting the Emulator

The environment is initialized using Docker Compose. The following command builds the containers and brings the network live:

```
cd Labsetup
docker-compose build
docker-compose up -d
```

2.2 Accessing the Network Map

The emulator provides a visual map at <http://localhost:8080/map.html>. This allows us to see the live status of BGP peering and traffic flows.



The screenshot shows the SEEDEMU Dashboard in a Firefox Web Browser. The browser address bar shows 'localhost:8080'. The dashboard title is 'SEEDEMU Dashboard' with a 'Help' link. The main section is titled 'Networks'. Below the title are buttons: 'Select All', 'Invert Selections', 'Deselect All', 'Delete selected...', '+ New network...', and 'Reload'. There is also a 'Search:' input field. The main content is a table with columns: 'ASN (scope)', 'Name', 'Type', 'Network Prefix', and 'Actions'. The table lists 10 entries, each with a checkbox in the 'ASN (scope)' column. The entries are:

ASN (scope)	Name	Type	Network Prefix	Actions
☐ 11	net_102_105	local	10.11.0.0/24	
☐ 12	net_101_104	local	10.12.0.0/24	
☐ 150	net0	local	10.150.0.0/24	
☐ 151	net0	local	10.151.0.0/24	
☐ 152	net0	local	10.152.0.0/24	
☐ 153	net0	local	10.153.0.0/24	
☐ 154	net0	local	10.154.0.0/24	
☐ 155	net0	local	10.155.0.0/24	
☐ 156	net0	local	10.156.0.0/24	
☐ 160	net0	local	10.160.0.0/24	

Showing 1 to 10 of 26 entries

Figure 1: Screenshot of the SEED Internet Emulator web map showing the AS connections.

3 Task 1: Stub Autonomous System

3.1 Task 1.a: Understanding AS-155's BGP Configuration

3.1.1 Task 1.a.1: Identifying AS-155's Peers

We access AS-155's BGP router container and inspect its BIRD configuration:

AS-155 has three BGP peering sessions, all on the IX-102 exchange network (10.102.0.0/24):

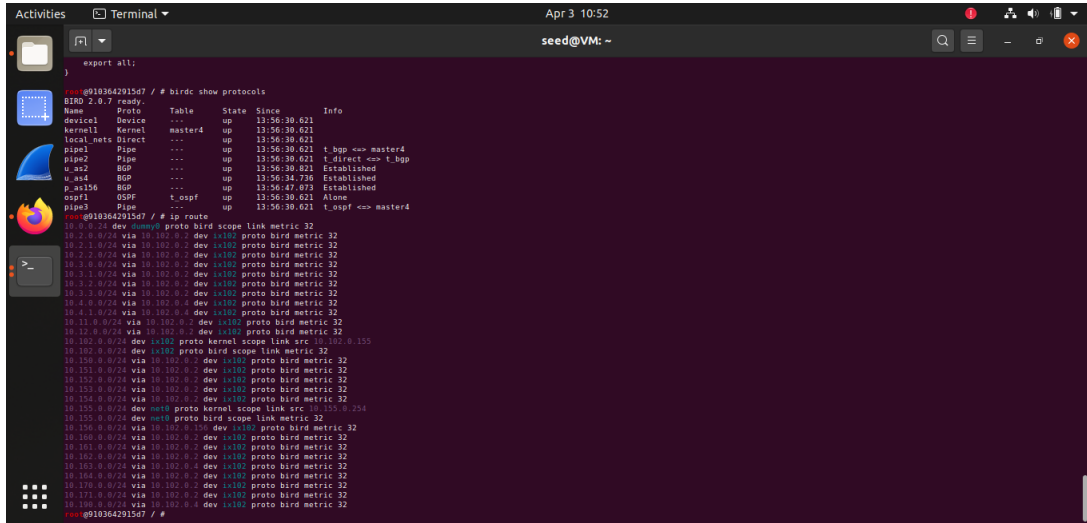
Protocol	Neighbor IP	Peer ASN	Relationship	Local Pref
u_as2	10.102.0.2	AS-2	Provider (upstream)	10
u_as4	10.102.0.4	AS-4	Provider (upstream)	10
p_as156	10.102.0.156	AS-156	Peer	20

Table 1: AS-155 BGP peering sessions derived from `bird.conf`.

3.1.2 Task 1.a.2: Demonstrating Redundancy via Failover

AS-155 peers with two upstream providers (AS-2 and AS-4), providing redundancy. We demonstrate this by disabling one provider session and observing that connectivity is maintained through the other.

Step 1 — Baseline routing table:



```
export all;
}

root@103642915d7: /# birdc show protocols
BIRD 2.0.7 ready.
Name Proto Table State Since Info
device1 Device --- up 13:56:30.621
kernel1 Kernel master4 up 13:56:30.621
local net1 Birect up 13:56:30.621
pipe1 Pipe --- up 13:56:30.621 t_bgp <=> master4
pipe2 Pipe --- up 13:56:30.621 t_direct <=> t_bgp
u_as2 BGP --- up 13:56:30.621 Established
u_as4 BGP --- up 13:56:34.736 Established
p_as156 BGP --- up 13:56:47.018 Established
ospf1 OSPF t_ospf up 13:56:30.621 Alone
pipe3 Pipe --- up 13:56:30.621 t_ospf <=> master4

root@103642915d7: /# ip route
10.0.0.0/24 dev dummy0 proto bird scope Link metric 32
10.2.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.2.1.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.2.2.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.3.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.3.1.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.3.2.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.3.3.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.4.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.4.1.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.11.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.12.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.102.0.0/24 dev ix102 proto kernel scope Link src 10.102.0.155
10.102.0.0/24 dev ix102 proto bird scope Link metric 32
10.150.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.151.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.152.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.153.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.154.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.155.0.0/24 dev net1 proto kernel scope Link src 10.155.0.254
10.155.0.0/24 dev net1 proto bird scope Link metric 32
10.156.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.158.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.161.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.162.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.162.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.164.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.170.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.171.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.176.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.177.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.178.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.179.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.180.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
root@103642915d7: /#
```

Figure 2: Baseline routing table on AS-155's router before any changes. Routes to remote prefixes are reachable via both AS-2 and AS-4.

Step 2 — Disable peering with AS-2:

```

Activities Terminal Apr 3 10:54 seed@VM: ~
10.170.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.171.0.0/24 via 10.102.0.2 dev ix102 proto bird metric 32
10.190.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
root@9103642915d7 / # birdc disable u_as2
BIRD 2.0.7 ready.
u_as2: disabled
root@9103642915d7 / # birdc show protocols
BIRD 2.0.7 ready.
Name Proto Table State Since Info
device1 Device --- up 13:56:30.621
kernel1 Kernel master4 up 13:56:30.621
local_nets Direct --- up 13:56:30.621
pipe1 Pipe --- up 13:56:30.621 t_bgp <=> master4
pipe2 Pipe --- up 13:56:30.621 t_direct <=> t_bgp
u_as2 BGP --- down 14:53:11.591
u_as4 BGP --- up 13:56:34.736 Established
p_as156 BGP --- up 13:56:47.073 Established
ospf1 OSPF t_ospf up 13:56:30.621 Alone
pipe3 Pipe --- up 13:56:30.621 t_ospf <=> master4
root@9103642915d7 / #

```

Figure 3: BGP protocol table after disabling u_as2. The session state changes from Established to down.

```

Activities Terminal Apr 3 10:56 seed@VM: ~
ospf1 OSPF t_ospf up 13:56:30.621 Alone
pipe3 Pipe --- up 13:56:30.621 t_ospf <=> master4
root@9103642915d7 / # ip route
10.0.0.0/24 dev dummy0 proto bird scope link metric 32
10.2.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.2.1.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.2.2.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.3.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.3.1.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.3.2.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.3.3.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.4.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.4.1.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.11.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.12.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.102.0.0/24 dev ix102 proto kernel scope link src 10.102.0.155
10.102.0.0/24 dev ix102 proto bird scope link metric 32
10.150.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.151.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.152.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.153.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.154.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.155.0.0/24 dev net0 proto kernel scope link src 10.155.0.254
10.155.0.0/24 dev net0 proto bird scope link metric 32
10.156.0.0/24 via 10.102.0.156 dev ix102 proto bird metric 32
10.160.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.161.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.162.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.163.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.164.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.170.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.171.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
10.190.0.0/24 via 10.102.0.4 dev ix102 proto bird metric 32
root@9103642915d7 / #

```

Figure 4: Routing table after disabling AS-2. Routes previously learned via AS-2 are now replaced by equivalent routes through AS-4, demonstrating automatic failover.

Step 4 — Verify connectivity from a host in AS-155:

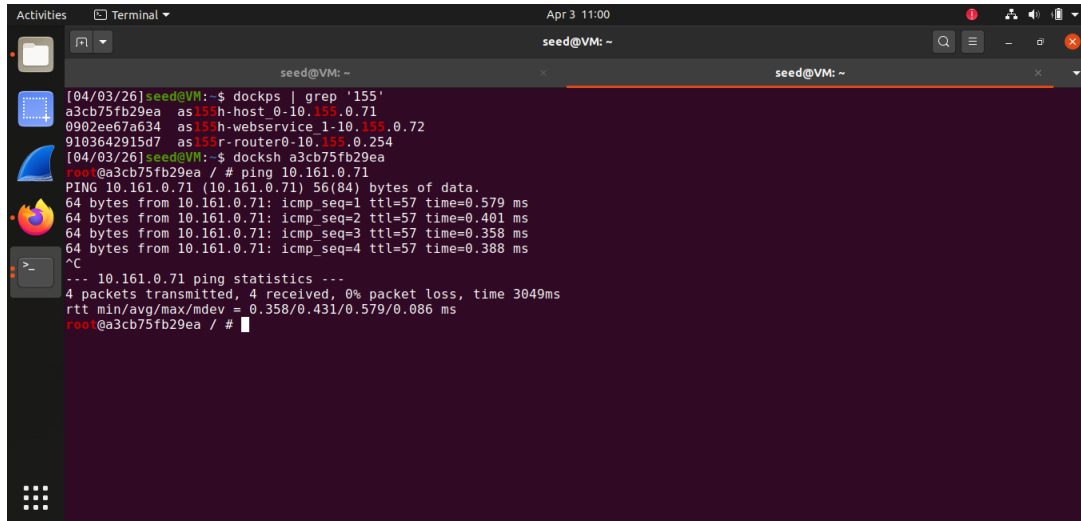


Figure 5: Ping to AS-161 host succeeds even with AS-2 disabled, confirming that AS-4 provides full redundancy as the backup upstream path.

Conclusion: Because AS-155 maintains two independent provider sessions, losing one provider does not disconnect AS-155 from the Internet. BIRD automatically reconverges and installs routes learned via the remaining provider.

3.2 Task 1.b: Observing BGP UPDATE Messages

To capture BGP UPDATE messages, we run `tcpdump` on AS-150's BGP router while triggering route changes from AS-155.

Triggering withdrawal and advertisement from AS-155:

```
# birdc disable u_as2 # triggers WITHDRAWAL message
# birdc enable u_as2  # triggers ADVERTISEMENT message
```

The captured packets are copied to the host and loaded into Wireshark.

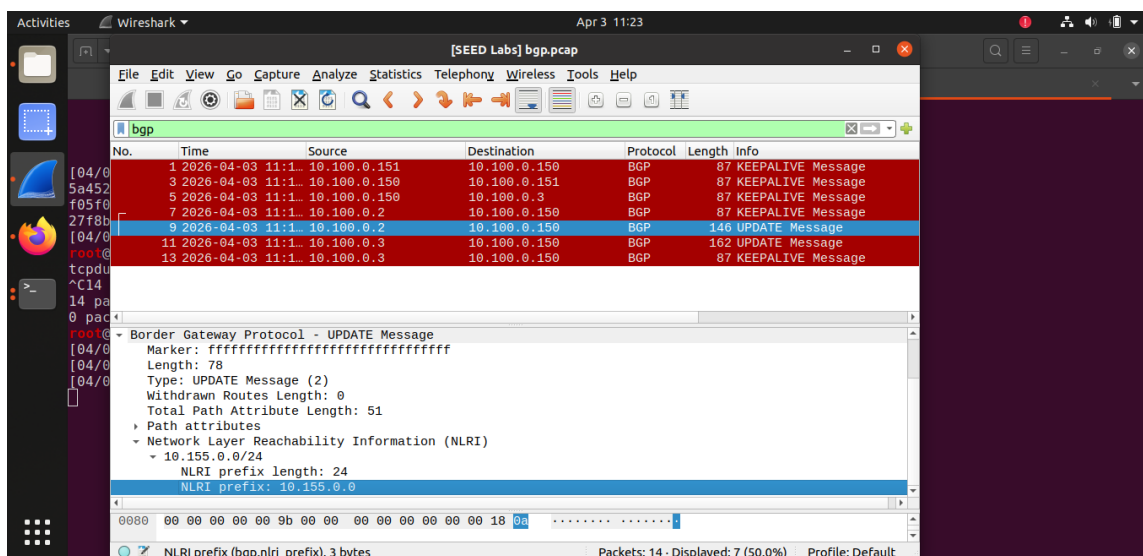


Figure 6: Wireshark capture of a BGP UPDATE message carrying a **Network Layer Reachability Information (NLRI)** field. This re-advertisement is triggered when the `u_as2` session is re-enabled, announcing the prefix as reachable again.

Analysis: A BGP UPDATE message serves a dual purpose. When a new route is advertised, the Total Path Attribute Length is non-zero, carrying attributes such as AS_PATH, NEXT_HOP, and LOCAL_PREF, followed by the NLRI field listing the newly reachable prefixes.

3.3 Task 1.c: Experimenting with Large Communities

Simulating the outage — disable AS-4 peering on AS-156:

```

[04/03/26]seed@VM:~$ dockerps|grep '156'
76bb2fcdab2a  as156h-webservice_1-10.156.0.72
08f15e057b43  as156h-host-0-10.156.0.71
ac8588ff838d  as156r-router0-10.156.0.254
[04/03/26]seed@VM:~$ docker exec 08f15e057b43
root@08f15e057b43 /# ping 10.155.0.71
PING 10.155.0.71 (10.155.0.71) 56(84) bytes of data.
64 bytes from 10.155.0.71: icmp_seq=1 ttl=62 time=0.343 ms
64 bytes from 10.155.0.71: icmp_seq=2 ttl=62 time=0.185 ms
^C
--- 10.155.0.71 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1032ms
rtt min/avg/max/mdev = 0.185/0.264/0.343/0.079 ms
root@08f15e057b43 /# ping 10.161.0.71
PING 10.161.0.71 (10.161.0.71) 56(84) bytes of data.
From 10.156.0.254 icmp_seq=1 Destination Net Unreachable
From 10.156.0.254 icmp_seq=2 Destination Net Unreachable
^C
--- 10.161.0.71 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1027ms
1 root@08f15e057b43 /#

```

Figure 7: After disabling the AS-4 session, pinging 10.155.0.71 (a direct peer) still succeeds, but 10.161.0.71 (a remote AS) returns *Destination Net Unreachable*. AS-155 does not forward traffic for AS-156 because their relationship is peer-to-peer, not provider-to-customer.

To allow AS-155 to act as a temporary provider for AS-156, two changes are made to AS-155's `bird.conf` for the `p_as156` block:

1. The import filter tags AS-156's routes as `CUSTOMER_COMM` (instead of `PEER_COMM`) and raises the local preference to 30, reflecting the new customer relationship.
2. The export filter is broadened to also share `PEER_COMM` and `PROVIDER_COMM` routes back to AS-156, giving it full Internet reachability through AS-155.

```

protocol bgp p_as156 {
  ipv4 {
    table t_bgp;
    import filter {
      bgp_large_community.add(CUSTOMER_COMM);    % was PEER_COMM
      bgp_local_pref = 30;                        % was 20
      accept;
    };
    export where bgp_large_community ~
      [LOCAL_COMM, CUSTOMER_COMM, PEER_COMM, PROVIDER_COMM];
    next hop self;
  };
  local 10.102.0.155 as 155;
  neighbor 10.102.0.156 as 156;
}

```

```

[04/03/26]seed@VM:~$ docker cp $(docker ps | grep as155r | awk '{print $1}'):etc/bird/bird.conf ./as155_bird.conf
[04/03/26]seed@VM:~$ nano as155_bird.conf
[04/03/26]seed@VM:~$ docker cp ./as155_bird.conf $(docker ps | grep as155r | awk '{print $1}'):etc/bird/bird.conf
[04/03/26]seed@VM:~$ docker exec -it $(docker ps | grep as155r | awk '{print $1}') birdc configure
BIRD 2.0.7 ready.
Reading configuration from /etc/bird/bird.conf
Reconfigured
[04/03/26]seed@VM:~$ docksh 08f15e057b43
root@08f15e057b43 / # ping 10.161.0.71
PING 10.161.0.71 (10.161.0.71) 56(84) bytes of data.
64 bytes from 10.161.0.71: icmp_seq=1 ttl=56 time=1.12 ms
^C
--- 10.161.0.71 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.124/1.124/1.124/0.000 ms
root@08f15e057b43 / # ping 10.155.0.71
PING 10.155.0.71 (10.155.0.71) 56(84) bytes of data.
64 bytes from 10.155.0.71: icmp_seq=1 ttl=62 time=0.256 ms
^C
--- 10.155.0.71 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.256/0.256/0.256/0.000 ms
root@08f15e057b43 / #

```

Figure 8: After reconfiguring AS-155, pinging 10.161.0.71 from AS-156’s host now succeeds. AS-155 is now forwarding AS-156’s traffic to its upstream providers AS-2 and AS-4, acting as a temporary transit provider.

3.4 Task 1.d: Configuring AS-180

AS-180 exists in the emulator but has no BGP peering configured. It connects to IX-105 (Houston). Our goal is to:

- Establish a **peer** relationship with AS-171.
- Establish **provider** relationships with AS-2 and AS-3 for full Internet access.

The following BGP blocks were added to AS-180’s `bird.conf`:

```

# Peer with AS-171
protocol bgp p_as171 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(PEER_COMM);
            bgp_local_pref = 20;
            accept;
        };
        export where bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM];
        next hop self;
    };
    local 10.105.0.180 as 180;
    neighbor 10.105.0.171 as 171;
}

# Provider: AS-2
protocol bgp u_as2 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(PROVIDER_COMM);
            bgp_local_pref = 10;
            accept;
        };
        export where bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM];
    };
}

```

```

        next hop self;
    };
    local 10.105.0.180 as 180;
    neighbor 10.105.0.2 as 2;
}

# Provider: AS-3
protocol bgp u_as3 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(PROVIDER_COMM);
            bgp_local_pref = 10;
            accept;
        };
        export where bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM];
        next hop self;
    };
    local 10.105.0.180 as 180;
    neighbor 10.105.0.3 as 3;
}

```

Matching peering blocks were also added to AS-2, AS-3, and AS-171's routers at IX-105. After reloading all configurations with `birdc configure`:

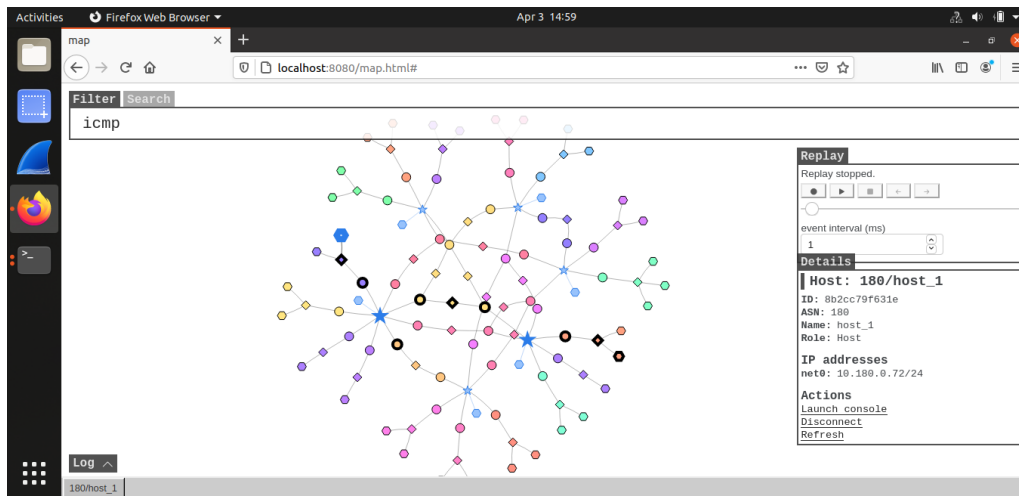


Figure 9: Route from AS-180's host to AS-171 (10.171.0.71) shows a direct path across IX-105, confirming the peer relationship is working correctly.

4 Task 2: Transit Autonomous System

4.1 Task 2.a: Experimenting with IBGP

Transit AS-3 uses IBGP to distribute externally learned routes among its internal routers at different Internet Exchanges. We observe the effect of disabling an IBGP session on AS-3's router at IX-103.

Baseline — routing table before disabling IBGP:

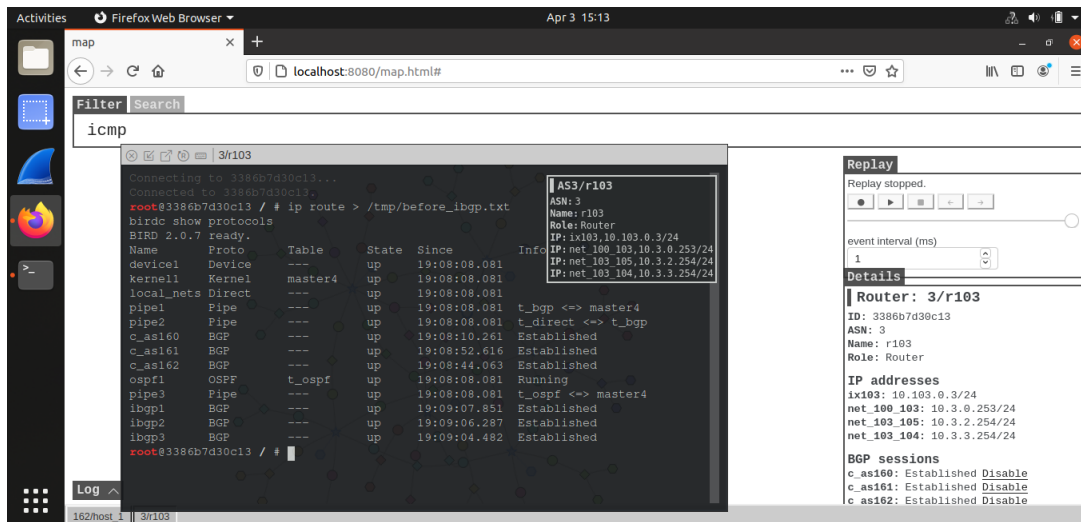


Figure 10: Routing table on AS-3's IX-103 router before any changes. Routes to remote prefixes are present, learned via IBGP from AS-3's other routers at IX-101 and IX-105.

Disable IBGP session:

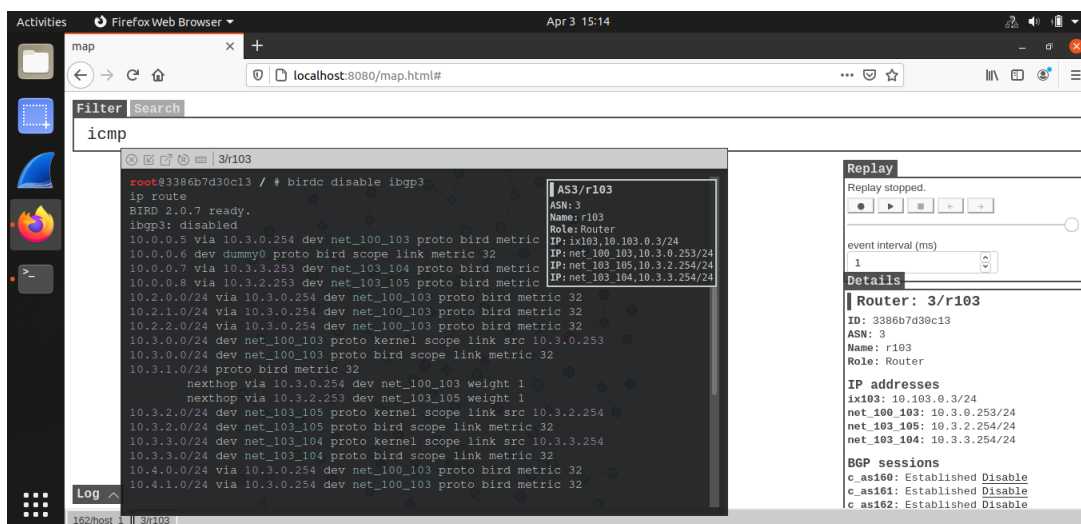


Figure 11: Routing table after disabling ibgp3. Routes that were learned from the disconnected IBGP peer disappear, reducing the router's visibility of the network.

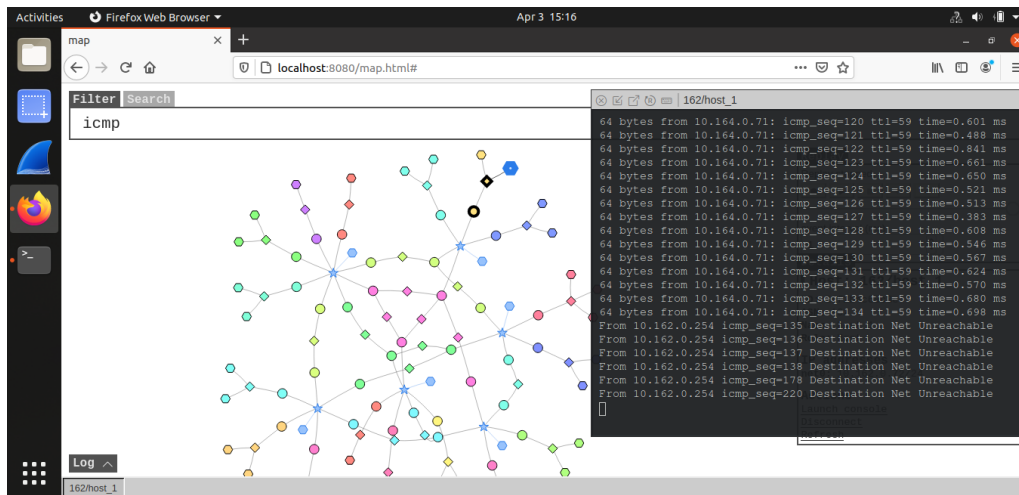


Figure 12: Ping from AS-162's host to 10.164.0.71 fails after the IBGP session is disabled. Without IBGP, AS-3's IX-103 router cannot reach prefixes learned at other IXes within AS-3.

Analysis: IBGP is essential for distributing externally learned routes between routers *within* the same AS. Without it, each internal router only knows about prefixes reachable through its own EBGP sessions, making transit impossible across the AS.

4.2 Task 2.b: Experimenting with IGP (OSPF)

While IBGP distributes BGP routes, IGP (OSPF here) provides the *next-hop reachability* information that IBGP depends on.

Re-enable IBGP, then record the routing table:

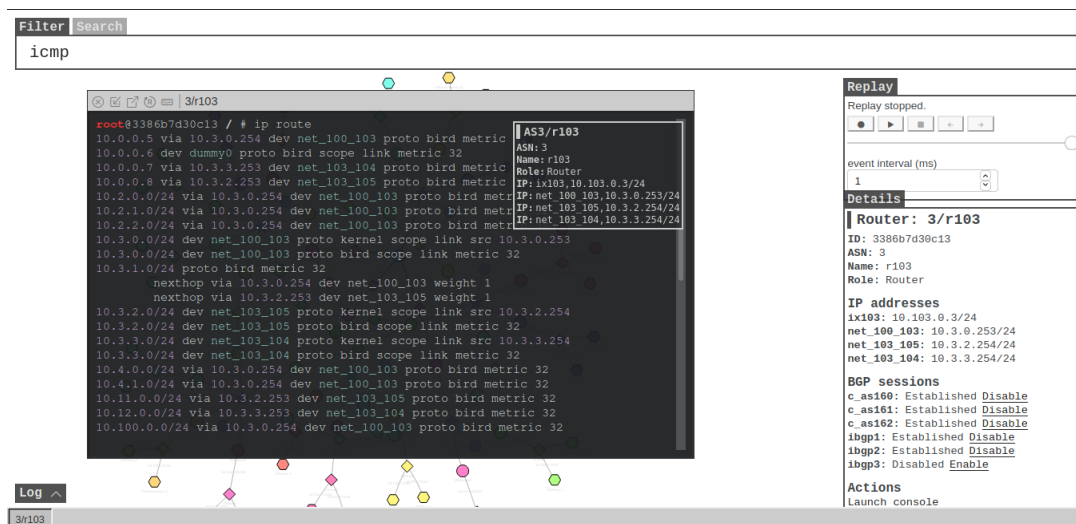


Figure 13: Routing table with IBGP and OSPF both active. Internal next-hop addresses are resolved via OSPF routes.

Disable OSPF:

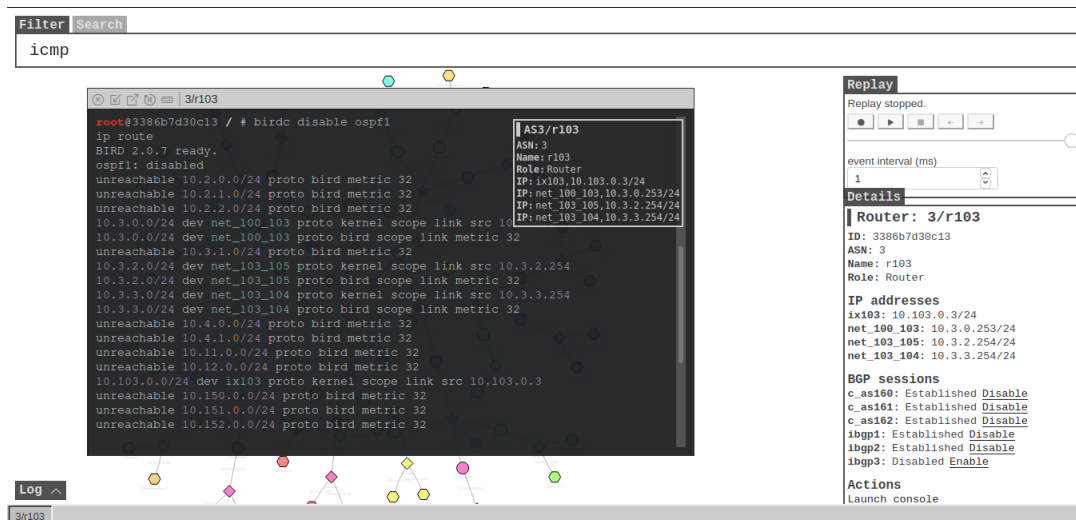


Figure 14: After disabling OSPF, internal routes disappear from the routing table. IBGP next-hops become unreachable because there is no IGP to resolve them.

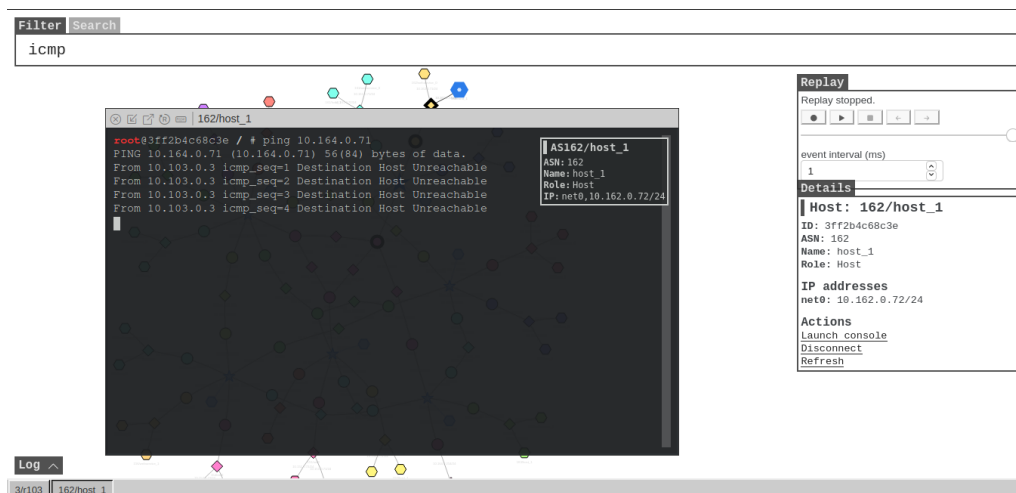


Figure 15: Connectivity test confirming packet loss after OSPF is disabled. Even though IBGP sessions remain up, packets cannot be forwarded without IGP-provided internal routes.

Analysis: IGP is indispensable in a transit AS because IBGP does not redistribute routes into the IGP. The next-hop IP addresses carried in IBGP UPDATE messages refer to *internal* routers whose addresses are only reachable via OSPF. Without OSPF, the router cannot forward packets even if it knows the BGP destination.

4.3 Task 2.c: Configuring AS-5

AS-5 is a transit AS present in the emulator but with no EBGP peering. It connects to three IXes: IX-101, IX-103, and IX-105. We configure it to provide transit to AS-153, AS-160, and AS-171, and establish a peer relationship with AS-3 at IX-103.

4.3.1 IBGP Configuration (Existing)

AS-5's IBGP is pre-configured. On the IX-101 router, the IBGP blocks look like:

```
protocol bgp ibgp1 {
```

```

    ipv4 {
        table t_bgp;
        import all;
        export all;
        next hop self;
    };
    local as 5;
    neighbor <IX-103-router-IP> as 5;
}

```

IBGP uses `next hop self` to ensure the advertising router rewrites the next-hop to itself, since IBGP peers are not on the same subnet.

4.3.2 EBGPeering Configuration

The following blocks were added to the respective AS-5 routers:

IX-101 router — Customer AS-153:

```

protocol bgp c_as153 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(CUSTOMER_COMM);
            bgp_local_pref = 30;
            accept;
        };
        export where bgp_large_community ~
            [LOCAL_COMM, CUSTOMER_COMM, PEER_COMM, PROVIDER_COMM];
        next hop self;
    };
    local 10.101.0.5 as 5;
    neighbor 10.101.0.153 as 153;
}

```

IX-103 router — Customer AS-160 and Peer AS-3:

```

protocol bgp c_as160 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(CUSTOMER_COMM);
            bgp_local_pref = 30;
            accept;
        };
        export where bgp_large_community ~
            [LOCAL_COMM, CUSTOMER_COMM, PEER_COMM, PROVIDER_COMM];
        next hop self;
    };
    local 10.103.0.5 as 5;
    neighbor 10.103.0.160 as 160;
}

protocol bgp p_as3 {
    ipv4 {
        table t_bgp;
        import filter {

```

```

        bgp_large_community.add(PEER_COMM);
        bgp_local_pref = 20;
        accept;
    };
    export where bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM];
    next hop self;
};
local 10.103.0.5 as 5;
neighbor 10.103.0.3 as 3;
}

```

IX-105 router — Customer AS-171:

```

protocol bgp c_as171 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(CUSTOMER_COMM);
            bgp_local_pref = 30;
            accept;
        };
        export where bgp_large_community ~
            [LOCAL_COMM, CUSTOMER_COMM, PEER_COMM, PROVIDER_COMM];
        next hop self;
    };
    local 10.105.0.5 as 5;
    neighbor 10.105.0.171 as 171;
}

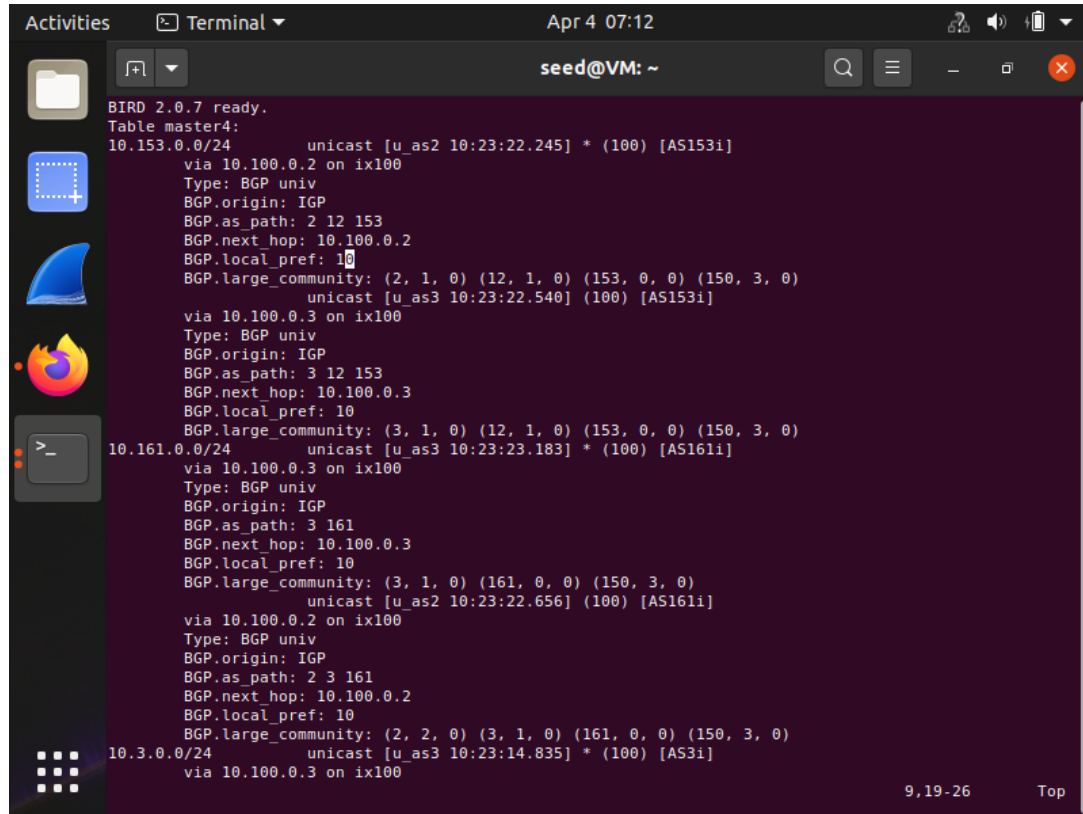
```

After pushing all configs with `export_bird_conf.sh` and running `birdc configure` on each router The connection status is verified.

5 Task 3: Path Selection

5.1 Task 3.a: Identifying the Best Route

On AS-150's BGP router, we dump all routes and compare them to the kernel routing table:



```
BIRD 2.0.7 ready.
Table master4:
10.153.0.0/24      unicast [u_as2 10:23:22.245] * (100) [AS153i]
                   via 10.100.0.2 on ix100
                   Type: BGP univ
                   BGP.origin: IGP
                   BGP.as_path: 2 12 153
                   BGP.next_hop: 10.100.0.2
                   BGP.local_pref: 10
                   BGP.large_community: (2, 1, 0) (12, 1, 0) (153, 0, 0) (150, 3, 0)
                   unicast [u_as3 10:23:22.540] (100) [AS153i]
                   via 10.100.0.3 on ix100
                   Type: BGP univ
                   BGP.origin: IGP
                   BGP.as_path: 3 12 153
                   BGP.next_hop: 10.100.0.3
                   BGP.local_pref: 10
                   BGP.large_community: (3, 1, 0) (12, 1, 0) (153, 0, 0) (150, 3, 0)
10.161.0.0/24      unicast [u_as3 10:23:23.183] * (100) [AS161i]
                   via 10.100.0.3 on ix100
                   Type: BGP univ
                   BGP.origin: IGP
                   BGP.as_path: 3 161
                   BGP.next_hop: 10.100.0.3
                   BGP.local_pref: 10
                   BGP.large_community: (3, 1, 0) (161, 0, 0) (150, 3, 0)
                   unicast [u_as2 10:23:22.656] (100) [AS161i]
                   via 10.100.0.2 on ix100
                   Type: BGP univ
                   BGP.origin: IGP
                   BGP.as_path: 2 3 161
                   BGP.next_hop: 10.100.0.2
                   BGP.local_pref: 10
                   BGP.large_community: (2, 2, 0) (3, 1, 0) (161, 0, 0) (150, 3, 0)
10.3.0.0/24        unicast [u_as3 10:23:14.835] * (100) [AS3i]
                   via 10.100.0.3 on ix100
```

Figure 16: BGP route table showing a prefix with multiple learned paths. Both routes have equal local preference (10), so BIRD selects the one with the shorter AS-Path as the best route.

```

BGP.as_path: 2 3 190
BGP.next_hop: 10.100.0.2
BGP.local_pref: 10
BGP.large_community: (2, 2, 0) (3, 1, 0) (190, 0, 0) (150, 3, 0)
root@27f8b41dead7 / #
root@27f8b41dead7 / # ip route
10.0.0.19 dev dummy0 proto bird scope link metric 32
10.2.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.2.1.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.2.2.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.3.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.3.1.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.3.2.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.3.3.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.4.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.4.1.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.11.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.12.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.100.0.0/24 dev ix100 proto kernel scope link src 10.100.0.150
10.100.0.0/24 dev ix100 proto bird scope link metric 32
10.150.0.0/24 dev net0 proto kernel scope link src 10.150.0.254
10.150.0.0/24 dev net0 proto bird scope link metric 32
10.151.0.0/24 via 10.100.0.151 dev ix100 proto bird metric 32
10.152.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.153.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.154.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.155.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.156.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.160.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.161.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.162.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.163.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.164.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.170.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
10.171.0.0/24 via 10.100.0.2 dev ix100 proto bird metric 32
10.190.0.0/24 via 10.100.0.3 dev ix100 proto bird metric 32
root@27f8b41dead7 / #

```

Figure 17: Kernel routing table confirming which route was installed. Only the best BGP route is passed to the kernel.

Analysis: BIRD's selection criteria are applied in order: (1) highest Local Preference wins; (2) if tied, shortest AS-Path wins. Since both providers assign local pref 10 to AS-150, the shorter AS-Path determines the winner.

5.2 Task 3.b: Preferring AS-3 over AS-2

To make AS-3 the primary link and AS-2 a backup, we raise the local preference of routes received from AS-3:

```

protocol bgp u_as3 {
  ipv4 {
    import filter {
      bgp_large_community.add(PROVIDER_COMM);
      bgp_local_pref = 20;  % raised from 10
      accept;
    };
    ...
  };
}

```

```

Activities  Terminal  Apr 4 07:16
seed@VM: ~
BIRD 2.0.7 ready.
Table master4:
10.153.0.0/24      unicast [u_as3 11:13:40.942] * (100) [AS1531]
via 10.100.0.3 on ix100
Type: BGP univ
BGP.origin: IGP
BGP.as_path: 3 12 153
BGP.next_hop: 10.100.0.3
BGP.local_pref: 20
BGP.large_community: (3, 1, 0) (12, 1, 0) (153, 0, 0) (150, 3, 0)
unicast [u_as2 10:23:22.245] (100) [AS1531]
via 10.100.0.2 on ix100
Type: BGP univ
BGP.origin: IGP
BGP.as_path: 2 12 153
BGP.next_hop: 10.100.0.2
BGP.local_pref: 10
BGP.large_community: (2, 1, 0) (12, 1, 0) (153, 0, 0) (150, 3, 0)
10.161.0.0/24      unicast [u_as3 11:13:40.942] * (100) [AS1611]
via 10.100.0.3 on ix100
Type: BGP univ
BGP.origin: IGP
BGP.as_path: 3 161
BGP.next_hop: 10.100.0.3
BGP.local_pref: 20
BGP.large_community: (3, 1, 0) (161, 0, 0) (150, 3, 0)
unicast [u_as2 10:23:22.656] (100) [AS1611]
via 10.100.0.2 on ix100
Type: BGP univ
BGP.origin: IGP
BGP.as_path: 2 3 161
BGP.next_hop: 10.100.0.2
BGP.local_pref: 10
BGP.large_community: (2, 2, 0) (3, 1, 0) (161, 0, 0) (150, 3, 0)
10.3.0.0/24        unicast [u_as3 11:13:40.942] * (100) [AS31]
via 10.100.0.3 on ix100
"route.after" 430L, 13212C
9,19-26  Top

```

Figure 18: Traceroute after the change confirms all traffic now exits AS-150 via AS-3. AS-2 remains available as a backup but receives no traffic while AS-3 is up.

6 Task 4: IP Anycast

AS-190 operates two physically separate networks that share the identical prefix 10.190.0.0/24 and host IP 10.190.0.100. BGP naturally routes each client to its topologically closest instance.

```

# From AS-150 host:
ping 10.190.0.100
# From AS-162 host:
ping 10.190.0.100

```

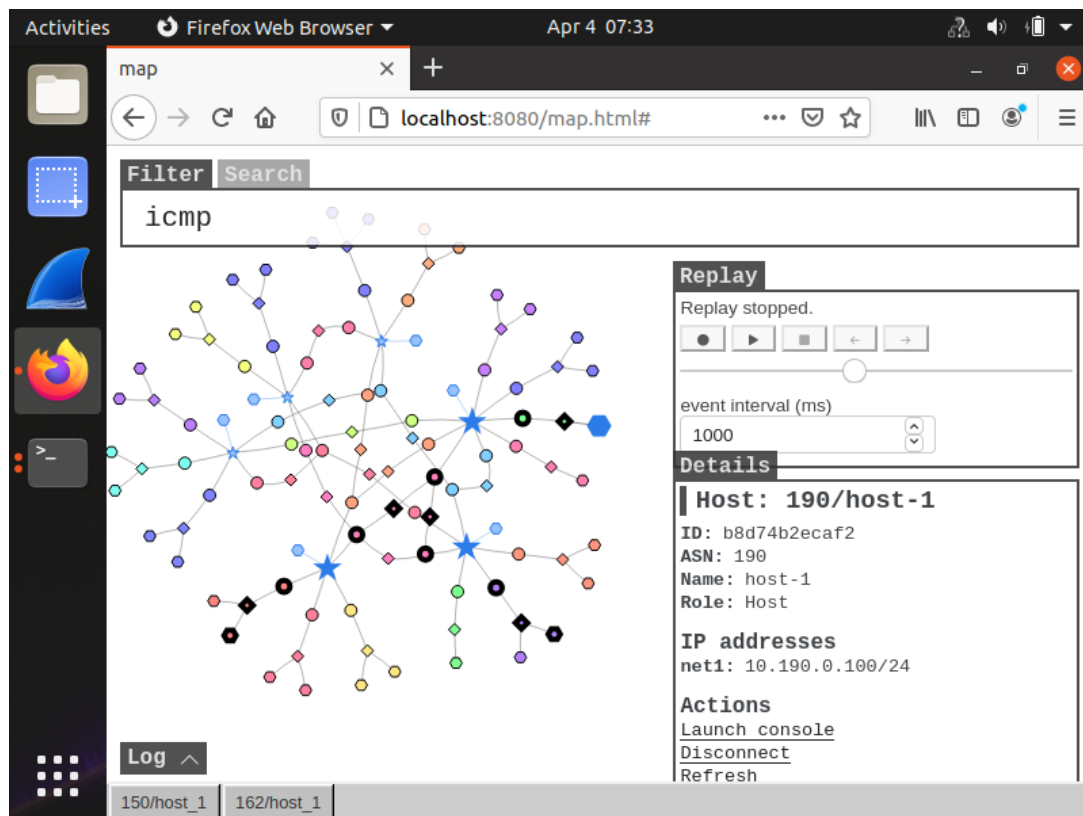



Figure 19: Emulator map with filter icmp and dst 10.190.0.100 showing two different physical paths to the same IP address, demonstrating BGP anycast in action.

Analysis: Each AS-190 router announces the same /24 prefix. Routers elsewhere select the announcement with the best BGP attributes (shorter AS-Path or higher local pref), which naturally corresponds to the geographically closer instance. This is the same mechanism used by DNS root servers globally.

7 Task 5: BGP Prefix Hijacking

7.1 Task 5.a: Launching the Attack from AS-161

We configure AS-161 to falsely announce AS-154's prefix using a static blackhole route tagged with LOCAL_COMM:

```
protocol static {
  ipv4 { table t_bgp; };
  route 10.154.0.0/24 blackhole {
    bgp_large_community.add(LOCAL_COMM);
  };
}
```

We also modify bgp u_as3 accordingly so that it prioritizes the new route :

```
protocol bgp u_as3 {
  ipv4 {
    table t_bgp;
    import filter {
```

```

    bgp_large_community.add(PROVIDER_COMM);
    bgp_local_pref = 10;
    accept;
};
export filter {
    # Explicitly allow the hijacked prefix to be sent to the neighbor
    if (net = 10.154.0.0/24) then accept;

    # Allow your normal local/customer routes
    if (bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM]) then accept;

    reject;
};
next hop self;
};
local 10.103.0.161 as 161;
neighbor 10.103.0.3 as 3;
}

```

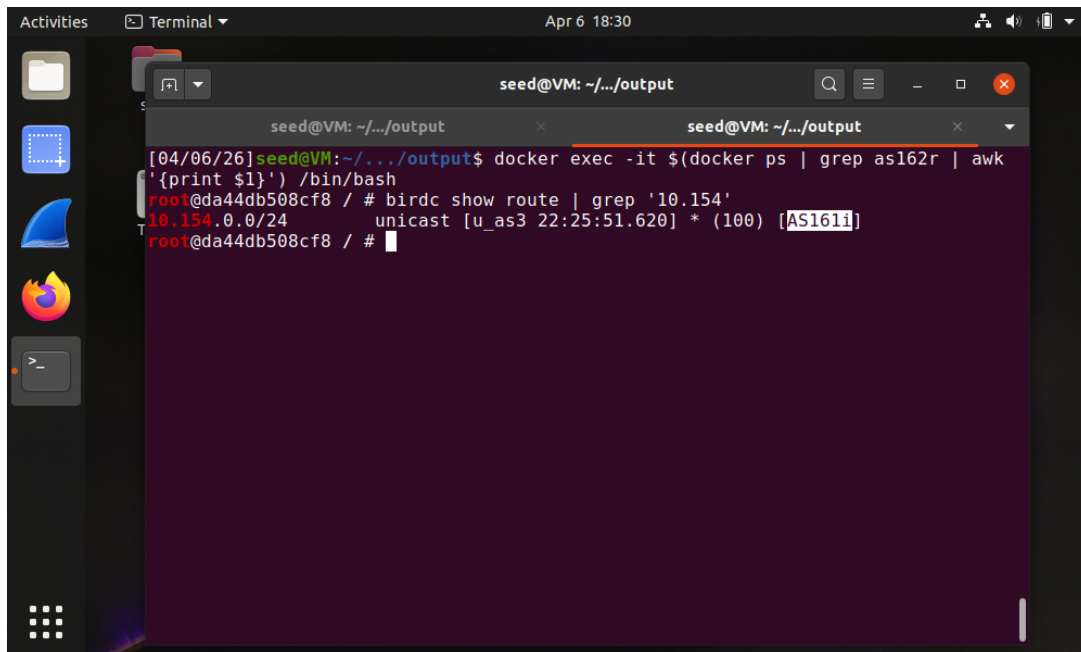


Figure 20: Route from AS-162 to 10.154.0.71 now terminates inside AS-161 rather than AS-154. The prefix has been successfully hijacked and traffic is blackholed.

Analysis: BGP has no built-in mechanism to verify prefix ownership. AS-161's announcement propagates freely because its upstream provider (AS-3) applies no filtering, and the announced path appears equally valid to all BGP routers.

7.2 Task 5.b: Fighting Back from AS-154

AS-154 reclaims its traffic by announcing two more-specific /25 prefixes. BGP's longest-prefix-match rule causes all routers to prefer these over AS-161's /24:

```

protocol static {
    ipv4 { table t_bgp; };
    route 10.154.0.0/25 blackhole {

```

```

        bgp_large_community.add(LOCAL_COMM);
    };
    route 10.154.0.128/25 blackhole {
        bgp_large_community.add(LOCAL_COMM);
    };
}

```

We also modify bgp u_as3 accordingly so that it prioritizes our route instead :

```

protocol bgp u_as2 {
    ipv4 {
        table t_bgp;
        import filter {
            bgp_large_community.add(PROVIDER_COMM);
            bgp_local_pref = 10;
            accept;
        };
        export filter {
            # Allow the more specific prefixes to be announced
            if (net = 10.154.0.0/25) then accept;
            if (net = 10.154.0.128/25) then accept;

            if (bgp_large_community ~ [LOCAL_COMM, CUSTOMER_COMM]) then accept;
            reject;
        };
        next hop self;
    };
    local 10.102.0.154 as 154;
    neighbor 10.102.0.2 as 2;
}

```

The screenshot shows a terminal window with the following content:

```

seed@VM: ~/.../output
/etc/bird/bird.conf:105:25 syntax error, unexpected UNICAST
root@e6328b4279eb / # vim /etc/bird/bird.conf
root@e6328b4279eb / # vim /etc/bird/bird.conf
root@e6328b4279eb / # birdc configure
BIRD 2.0.7 ready.
Reading configuration from /etc/bird/bird.conf
/etc/bird/bird.conf:105:25 syntax error, unexpected CF_SYM_UNDEFINED
root@e6328b4279eb / # vim /etc/bird/bird.conf
root@e6328b4279eb / # birdc configure
BIRD 2.0.7 ready.
Reading configuration from /etc/bird/bird.conf
Reconfigured
root@e6328b4279eb / # birdc show route | grep '10.154'
10.154.0.0/25      blackhole [static1 22:34:41.832] * (200)
10.154.0.0/24      unicast [local_nets 21:42:34.985] * (240)
10.154.0.128/25    blackhole [static1 22:34:41.832] * (200)
root@e6328b4279eb / # exit
[04/06/26]seed@VM:~/.../output$ docker exec -it $(docker ps | grep as162r | awk '{print $1}') /bin/bash
root@da44db508cf8 / # birdc show route | grep '10.154'
10.154.0.0/25      unicast [u_as3 22:34:41.996] * (100) [AS154i]
10.154.0.0/24      unicast [u_as3 22:25:51.620] * (100) [AS161i]
10.154.0.128/25    unicast [u_as3 22:34:41.996] * (100) [AS154i]
root@da44db508cf8 / #

```

Figure 21: Traceroute now correctly reaches AS-154 again. The more-specific /25 announcements override AS-161's /24 at every BGP router.

Analysis: AS-154 successfully regains its traffic by exploiting the Longest Prefix Match

algorithm. In IP routing, a router will always prefer the most specific route (the one with the longest subnet mask) in its routing table. By announcing two /25 prefixes, AS-154 provides a more specific path than AS-161's hijacked /24 advertisement. Consequently, all BGP routers in the topology prioritize the /25 routes, effectively neutralizing the hijack regardless of other BGP attributes like AS-Path length.

7.3 Task 5.c: Fixing the Problem at AS-3

AS-3 adds a filter to its import policy for AS-161, explicitly rejecting any announcement of AS-154's prefix:

```
protocol bgp c_as161 {
  ipv4 {
    import filter {
      if net = 10.154.0.0/24 then reject;
      bgp_large_community.add(CUSTOMER_COMM);
      bgp_local_pref = 30;
      accept;
    };
    ...
  };
}
```

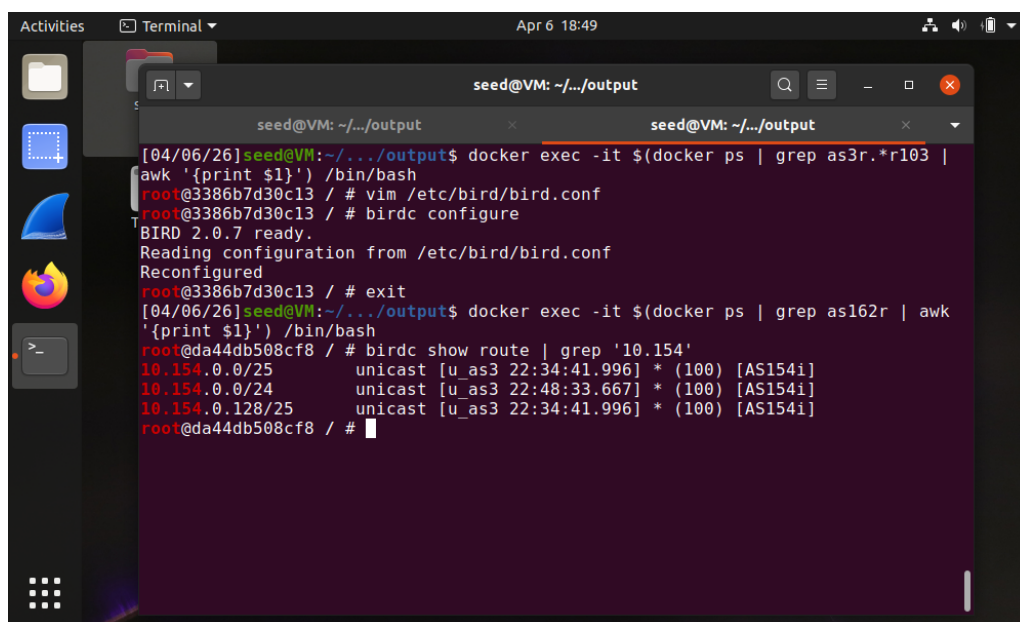


Figure 22: After AS-3 applies the filter and reloads its configuration, the BGP route table on AS-162 shows AS-154 as the correct origin of 10.154.0.0/24. AS-161's fake announcement no longer propagates beyond AS-3.

Analysis: Provider-level filtering (sometimes called RPKI or prefix filtering) is currently the most practical defense against BGP hijacking. By validating what prefixes a customer is allowed to announce, AS-3 prevents the attack from spreading while keeping the peering session intact so AS-161's legitimate users retain Internet access.

8 References

1. SEED Labs: BGP Exploration and Attack Lab Description.

2. RFC 4271: A Border Gateway Protocol 4 (BGP-4).
3. Wenliang Du, "Internet Security: A Hands-on Approach", 3rd Edition.